

Module 7: Automating Your Workflow with CI/CD

So far, we've designed, built, and refactored a complete application. We've tested it manually and then deployed it... well, we haven't yet! Every time we make a change, we have to manually run tests, build the project, and then copy the files over to a server. This process is slow, repetitive, and prone to human error.

What if we could automate it all? What if every time you pushed your code, a series of checks were automatically run to ensure it was high quality? And what if, after those checks passed, your application was deployed to your users without you lifting a finger?

This is the power of **CI/CD**. It stands for Continuous Integration and Continuous Delivery/Deployment. It's a cornerstone of modern software development that allows teams to deliver code more frequently and reliably. In this module, we'll dive into the concepts, benefits, and tools that make this automation possible.

Objectives and Key Results

Objective	Key Result
Understand the core principles of CI/CD.	You can clearly explain what Continuous Integration, Continuous Delivery, and Continuous Deployment are and why they are important.
Implement a basic CI pipeline.	You can use GitHub Actions to create a workflow that automatically runs checks (like linting or testing) on your code every time you push it.
Explore modern CI/CD tools.	You can identify different types of CI/CD tools and describe their roles in the development lifecycle (e.g., Husky for pre-commit, GitHub Actions for CI, Vercel/Render for CD).

7.1 What is CI/CD?

CI/CD is not a single tool, but a philosophy and a set of practices. It's composed of two main parts that are often executed in a sequence.

CI: Continuous Integration

Continuous Integration is the practice of automating the integration of code changes from

multiple contributors into a single software project.

- **What it is:** Imagine a team where everyone works on their own feature for weeks and then tries to merge it all together at the end. It's a nightmare of conflicting code! CI avoids this. Developers are encouraged to merge their changes back to the main branch frequently—often several times a day.
- **How it works:** Each time a developer pushes code to the repository, an automated process kicks in. This process automatically **builds** the application and runs a suite of **automated tests** against it.
- **The Goal:** To detect integration bugs early and quickly. If the build or tests fail, the team is notified immediately and can fix the issue before it becomes a bigger problem. A successful CI run means the code is, at a minimum, technically sound and doesn't break existing functionality.

CD: Continuous Delivery vs. Continuous Deployment

This is where the automation continues after a successful CI build. There are two "flavors" of CD.

1. **Continuous Delivery** This is the practice of ensuring your code is *always* in a deployable state. After the CI stage, the process continues by automatically preparing a release build and deploying it to a testing or staging environment. The final step—deploying to production—is triggered by a **manual button push**.
 - **The Goal:** To reduce the risk of release. Because you have a fully tested, ready-to-go build, you can decide to release it to your users at any time with confidence.
2. **Continuous Deployment** This is the ultimate form of automation. It takes Continuous Delivery one step further. Every change that passes all the automated tests in the CI/CD pipeline is **automatically deployed to production**. There is no manual intervention.
 - **The Goal:** To accelerate the feedback loop with users. You can get new features and bug fixes into the hands of your users as soon as they are ready. This is used by companies like Netflix and Amazon to deploy thousands of times per day.

7.2 What is CI/CD Used For? The Benefits vs. The Drawbacks

CI/CD is primarily used to:

- **Automate Everything:** From code style checks and testing to building and deploying.
- **Improve Code Quality:** By running automated checks on every single change, it's much harder for bugs or bad code to slip through.
- **Increase Development Speed:** Automation frees developers from slow, manual tasks. It enables smaller, more frequent releases, getting features to users faster.
- **Reduce Risk:** By testing small changes individually, the risk of a major failure in production is significantly lower. If a bug does occur, it's easier to pinpoint which small change caused it.

Benefits 	Drawbacks 
Faster Release Cycles: Ship features and fixes to users in minutes, not weeks.	Initial Setup Complexity: Designing and implementing a robust pipeline requires time,

	planning, and expertise.
Improved Developer Productivity: Developers can focus on writing code instead of getting bogged down in manual deployment processes.	Requires a Testing Culture: CI/CD is only as good as your automated tests. "Garbage in, garbage out."
More Reliable Builds: Automated processes are consistent and repeatable, eliminating human error.	Maintenance Overhead: As your application grows, your CI/CD pipeline also needs to be maintained and updated.
Early Bug Detection: Catching issues moments after they're introduced is far cheaper and easier than finding them in production.	False Sense of Security: A "green" pipeline doesn't guarantee the application is perfect, only that it passed the defined checks.

7.3 The Tools of the Trade: Kinds of CI/CD

There are countless tools available, each fitting into a different part of the workflow.

Kinds of CI (Running checks)

- **Pre-commit Hooks (e.g., Husky):** This is your first line of defense. These tools run scripts on your local machine *before* you're even allowed to commit your code. This is perfect for running linters (to check code style) or fast unit tests. It prevents simple mistakes from ever reaching the repository.
- **CI Platforms (e.g., GitHub Actions, GitLab CI, Jenkins):** These are the engines of your CI process. They are platforms that watch your code repository. When they detect a change (like a git push), they spin up a temporary server (a "runner"), pull your code, and execute a series of steps you've defined in a configuration file. GitHub Actions is built directly into GitHub and is one of the most popular and accessible options today.

Kinds of CD (Deploying the code)

- **Platform Integrations (e.g., Vercel, Netlify, Render, Heroku):** This is the easiest way to achieve Continuous Deployment. These hosting platforms connect directly to your GitHub (or other Git provider) repository. You configure them once, and from then on, every time you push to your main branch, they automatically pull the code, build it, and deploy the new version. Vercel and Netlify are famous for front-end, while **Render** and Heroku are fantastic for back-end services like our Node.js API.
- **CI Platform Extensions (e.g., GitHub Actions for Deployment):** You can write more advanced scripts within your CI platform to handle deployment. For example, after your tests pass in GitHub Actions, you can add more steps to securely log into a cloud provider (like AWS or Google Cloud) and deploy your application. This offers more control and flexibility.
- **Webhooks:** This is a more traditional method where your Git provider sends a simple HTTP request (a webhook) to a specific URL on your server whenever there's a push. Your server listens for this webhook and, upon receiving it, runs a script to pull the latest code and restart itself.

7.4 How to Do Simple CI/CD: A GitHub Actions Example

Let's see what a simple CI configuration looks like. With GitHub Actions, you define your CI "workflow" in a YAML file. This file lives in a special directory in your project:

`.github/workflows/`.

Here's a simple workflow for our Node.js Blog API. This workflow will:

1. Run automatically on every push to the main branch.
2. Set up a clean environment with a specific version of Node.js.
3. Install our project's dependencies.
4. Run a linter to check for code style issues.

```
# .github/workflows/ci.yml

# Name of the workflow
name: Node.js CI

# Controls when the workflow will run
on:
  # Triggers the workflow on push events for the "main" branch
  push:
    branches: [ "main" ]
  # Also triggers on pull requests targeting the "main" branch
  pull_request:
    branches: [ "main" ]

# A workflow run is made up of one or more jobs that can run
# sequentially or in parallel
jobs:
  # This workflow contains a single job called "build"
  build:
    # The type of runner that the job will run on
    runs-on: ubuntu-latest

    # Steps represent a sequence of tasks that will be executed as
    # part of the job
    steps:
      # Step 1: Checks-out your repository under $GITHUB_WORKSPACE,
      # so your job can access it
      - name: Checkout repository
        uses: actions/checkout@v3

      # Step 2: Sets up a specific Node.js version
```

```

- name: Use Node.js 18.x
  uses: actions/setup-node@v3
  with:
    node-version: '18.x'
    cache: 'npm' # Caches npm packages for faster builds

# Step 3: Installs dependencies. 'ci' is often preferred over
'install' in CI
# because it uses the package-lock.json for a deterministic
build.
- name: Install dependencies
  run: npm ci

# Step 4: Runs the linter. This command must be defined in your
package.json
- name: Run linter
  run: npm run lint

```

Your Task

It's time to add a professional CI/CD workflow to the project you built and refactored in Modules 4 and 5. This will ensure code quality automatically and set you up for seamless deployments.

Part 1: Pre-commit Hooks with Husky (Local CI)

First, let's stop bad code from ever being committed. We'll use Husky to run a linter before each commit.

1. **Install Dependencies:** In your project terminal, install ESLint (if you haven't already), Husky, and lint-staged.


```

# Install a linter
npm install --save-dev eslint
npx eslint --init # Go through the setup process

# Install Husky and lint-staged
npm install --save-dev husky lint-staged

```
2. **Activate Hooks:** Enable Git hooks with Husky.


```

npx husky install

```
3. **Create a Hook:** Create a pre-commit hook that will run lint-staged.


```

npx husky add .husky/pre-commit "npx lint-staged"

```
4. **Configure lint-staged:** In your package.json, add a lint-staged key. This tells it to run

eslint on any JavaScript files that are part of the commit.

```
// package.json
{
  // ... your other configurations
  // Modify this as you needed
  "lint-staged": {
    "*.js": "eslint --fix"
  }
}
```

5. **Test It:** Make a small, deliberate style error in one of your files (e.g., remove a semicolon). Now, try to git add . and git commit -m "test". Your commit should be **rejected** by Husky, and ESLint will report the error!
6. Make a documentation of your working husky in a docs (success and error cases)

Part 2: CI with GitHub Actions (Remote CI)

Now, let's create a workflow that runs on GitHub's servers.

1. **Create the Workflow File:** In the root of your project, create the directory and file: `.github/workflows/ci.yml`.
2. **Add the Workflow Code:** Copy the YAML example from section 7.4 into your `ci.yml` file.
3. **Add a Lint Script:** Make sure your `package.json` has a lint script that GitHub Actions can run.

```
// package.json
"scripts": {
  "start": "node app.js",
  "lint": "eslint . --ext .js", // Add this line
  // ... other scripts
},
```

4. **Commit and Push:** Commit all your new files (`.husky/`, `.github/`, and your updated `package.json`) and push them to your GitHub repository.
5. **Observe the Magic:** Go to your repository on GitHub and click on the **"Actions"** tab. You will see your "Node.js CI" workflow running. Click on it to see the live logs of each step. If all goes well, you'll see a green checkmark!
6. Make a documentation of working github action.

Part 3 (Bonus): Continuous Deployment with Render

Let's complete the loop and deploy our API for free.

1. **Sign up:** Create a free account on [Render.com](https://render.com) using your GitHub account.
2. **Create a New Web Service:** On the Render dashboard, click "New +" -> "Web Service".
3. **Connect Your Repo:** Choose the Blog API repository from your GitHub account.
4. **Configure the Service:**
 - o **Name:** Give your service a unique name (e.g., `my-blog-api`).

- **Region:** Choose a region close to you.
 - **Branch:** Select main.
 - **Runtime:** Select Node.
 - **Build Command:** npm install
 - **Start Command:** npm start (or whatever command starts your Express server).
5. **Add Environment Variables:** This is crucial. Click on "Advanced" and add the environment variables your application needs, such as your DATABASE_URL and JWT_SECRET. **Do not commit these secrets to your code.**
 6. **Deploy:** Click "Create Web Service". Render will pull your code, run the build command, and start your server. It will give you a public URL (e.g., <https://my-blog-api.onrender.com>).
 7. Make a documentation of automatic deployment and provide ur deployed application url.

Now, your full CI/CD pipeline is active. Every time you git push to your main branch, GitHub Actions will run your checks, and if they pass, Render will automatically deploy the new version of your API. Congratulations, you've automated your workflow!